Prática — Troca de chaves Diffie-Hellman

DCC075 – Segurança em Sistemas Computacionais · UFJF · Lucas Gonçalves Rocha · Matrícula 202265187C · Individual · Outubro de 2026

1. Valores utilizados

Variável	Papel	Origem
p	público	primo de 2048 bits do RFC 3526 (grupo 14), usado em TLS, SSH e IPsec; primo seguro
g = 11	público	menor raiz primitiva módulo p, encontrada e verificada pelo código
a, **b**	privados (Alice, Bob)	2048 bits, sorteados com gerador criptográfico em [2, p−2]
A, **B**	trafegam no canal público	$A = g^a \bmod p$; $B = g^b \bmod p$
s	segredo (nunca trafega)	$s = B^a \bmod p = A^b \bmod p$; conferido com uma implementação independente em Python

Valores completos, em hexadecimal (exceto g):

```
p (RFC 3526, grupo 14, 2048 bits):
FFFFFFFF FFFFFFFF C90FDAA2 2168C234 C4C6628B 80DC1CD1 29024E08 8A67CC74
020BBEA6 3B139B22 514A0879 8E3404DD EF9519B3 CD3A431B 302B0A6D F25F1437
4FE1356D 6D51C245 E485B576 625E7EC6 F44C42E9 A637ED6B 0BFF5CB6 F406B7ED
EE386BFB 5A899FA5 AE9F2411 7C4B1FE6 49286651 ECE45B3D C2007CB8 A163BF05
98DA4836 1C55D39A 69163FA8 FD24CF5F 83655D23 DCA3AD96 1C62F356 208552BB
9ED52907 7096966D 670C354E 4ABC9804 F1746C08 CA18217C 32905E46 2E36CE3B
E39E772C 180E8603 9B2783A2 EC07A28F B5C55DF0 6F4C52C9 DE2BCBF6 95581718
3995497C EA956AE5 15D22618 98FA0510 15728E5A 8AACAA68 FFFFFFFF FFFFFFFF

g: 11 (decimal)

a (privada da Alice):
46690A49 B3CFFA43 E76D0ABD 5291B70E 09B57951 8D38B0D4 4583426D E345D2E2
D69FEF4D 7F843145 F063CE25 A860735F DA2ABF60 F1E5719C 90369C21 76990831
D599E69E AD0C4F89 4A21AE8C AC80F7ED 773C20F2 0B0F6F54 A32C1F5E 43F1B2D1
D7126D67 9F1102FF B6EC7087 22BA51A8 75D1D5EB 2179F228 55068A71 E6AEC44F
C6756FDA F2197684 709D5EAA 2458B409 63ECD7B1 2A6DC6AE 86689E2A A784EF99
23FE0709 B4049509 B7487E70 087FD164 2A02DE49 3CB938A5 532DBB93 FB44C71A
BEAFBC4D F221F0A2 AF30E2E2 2C61D06A 7894A586 A1ECE9FB FB4F0E2A 0C8103AD
A47EC896 2B1A487E C1F52DA4 FA25D139 42CBD115 D8B6D75F 488E0139 C429FC40

b (privada do Bob):
66D32BB1 097F781F 5DABBF5A D2396F2F DCC07BC0 29715AD3 039CB27E 6142F97B
1E708337 8D8B7317 AFBDC0C9 F6E28EB5 9DBE0F77 B54BA026 EDF596C3 C2BCD1AA
5F6F74C6 8FE73542 8F745A94 5E9FDBBC A5AECC6F 0119E696 7A70AE08 5BD3B701
3D22FFD8 DE1E501B CC33B854 3C7585B2 CFEF3C63 19A4A1B7 04563674 CE15D26B
3108D4E7 C2588526 07A6E8BC BFF45651 1440954D 153F4DAD D3D5CC41 56B0F29B
12CFFAA4 8E359992 E43A5A2C 12969392 16499095 4302B3D9 73829EE4 87982872
DDC0B4F0 5A77A109 6863AEB3 57BFEAFF 17AECACB 7A047592 1A73E6C5 C9695947
4062D905 D03294C0 6BF7CDD2 ACC3529C 0652231F B1C837D9 1569F728 DA5F427D

A = g^a mod p:
C1C275C1 405E4956 9529950D 465DB6C8 15BF7770 BDF816F9 EC39711A 21E8881B
42BFB196 56F6B671 13C0F341 C2E5D737 D0897307 2DEB114E 44C1EE8E 069D2629
2FF90D1C 60F8057F A8180683 318D2A23 70312AEA F9A1839D 5BF8CE20 741D12AA
E3FAD435 591C3A7F 07538DDA 58EBC759 A154D264 7FE4A680 03CE0B9B 9B3115C1
A21F5AED 2F6B4C64 0BC784EC 4E4765C4 D3CCF0DC 6013A154 8E9C03C3 76FDBACA
A8647E2A 21A13EB3 9F686186 8458876D BB54AD8B 66BF6230 09CAD0E8 DA2C4999
48972F86 9EDF35C6 03D63DA3 84B560A3 1B6D1127 38FC9619 DFA469CF A017C0A6
464F1E00 02291CA4 913CD4BC 25C37524 3BD1EB72 8658B714 C31547AC D2F0F16D

B = g^b mod p:
DFA64919 54CF32BF 05B1481E 68E2AD98 435A86C8 2E0E26CD E31C254C D5DBBA6C
FF9F8596 63CB9A6B 59822545 34FE7859 A57B9435 59FA037C 894FC18D 5BD1A198
3ADDD71A 1C0E64FB 6239BB5E DA69DC22 79B70D56 0A0C0E57 36A2F9EA AAF05B92
AA15C748 0DEABC1A 03DB03DA 95ACACFA 52708A47 5EFEC1BF FC2C0FEE BE3CD3AD
01CF2E18 768C73BE 12197AF8 3E29E0AA 4AEF70CC 7C71EF60 5BB74B55 B9214E03
65FC7265 D03BB9A1 BCA8C053 6D4660D3 958F6E9B 498D03B8 12C0D04D A98962E4
```

```
809A5EDC B1DC2479 BE2A52C9 10C01526 1766EAE4 BAC5DE46 10DF7941 C0A986AE
A9CDA818 A146EE25 B310A8C8 66158F8A 4BE35892 D17D7F81 2472FD3A E5D8664D

s = B^a mod p = A^b mod p:
E453D3C4 F7FBA95E 83822C80 F519E8A8 9345D30D 54B38A24 F4183C1B DFE283C5
7D4BDD28 AD2490A8 2F353A2A B33B1F98 4245808F 3D5CEC93 EE5EAC8D 59D5CDB0
952D4079 8194A29B F729438F F70C3E97 6194C2BD D2DD7627 AFE53740 2CCB80D4
90183EA6 CBE4EC32 CC0B75C6 D01152F2 8DB85D54 3371D053 08C1DDCD FB7B0003
C33F86C7 BE5F9F32 797ADD0E B282814D FDF4BDC4 259AFC3C 3F7C37EC ECED19DB
EEA8AD03 4756423A 24B540AC 23D698E1 94E2E6B3 6A26BBFC 3A4024E6 43B2056E
FFED47E6 96C3A019 3E8C06F4 2378B59B B53A9973 3F54E64F CC914F2D 3A2A20EF
7509BEC1 FEA17AB5 732BC13E F64BFB14 D1A45671 B745E356 E665A4E3 C9479028
```

2. O protocolo

Alice e Bob, que só conversam por um canal público, combinam um segredo sem nunca transmiti-lo:

```
Públicos: p (primo) e g (raiz primitiva mod p)
Alice sorteia a (secreto)  →  envia A = g^a mod p
Bob   sorteia b (secreto)  →  envia B = g^b mod p
Alice calcula s = B^a mod p = (g^b)^a = g^(ab) mod p
Bob   calcula s = A^b mod p = (g^a)^b = g^(ab) mod p     → o mesmo s
```

Quem escuta o canal vê p, g, A e B. Para calcular s, precisaria descobrir a a partir de $A = g^a \bmod p$ — o problema do logaritmo discreto, inviável com p de 2048 bits. Exemplo pequeno, feito à mão e usado nos testes automatizados (p = 23, g = 5, a = 6, b = 15): $A = 5^6 \bmod 23 = 8$; $B = 5^{15} \bmod 23 = 19$; Alice calcula $19^6 \bmod 23 = 2$ e Bob calcula $8^{15} \bmod 23 = 2$.

3. Raiz primitiva e primo seguro

g é raiz primitiva módulo p quando suas potências geram todos os valores de 1 a p−1, isto é, quando $g^{(p-1)/f} \not\equiv 1 \pmod p$ para todo fator primo f de p−1. Verificar isso exige fatorar p−1, o que é inviável para um p de 2048 bits qualquer. Por isso foi usado um **primo seguro**, p = 2q + 1 com q primo: os únicos fatores de p−1 são 2 e q, e o teste se reduz a $g^2 \not\equiv 1$ e $g^q \not\equiv 1 \pmod p$. O valor de p foi conferido com a fórmula do RFC 3526 ($p = 2^{2048} - 2^{1984} - 1 + 2^{64} \cdot (\lfloor 2^{1918} \cdot \pi \rfloor + 124476)$), e o código confirma, pelo teste de Miller-Rabin, que p e q são primos.

O gerador do próprio RFC é g = 2, mas ele **não é raiz primitiva**: $2^q \equiv 1 \pmod p$, então gera apenas o subgrupo de ordem q. Os valores de 2 a 10 também não são raízes primitivas; **11 é a menor**. O mesmo ocorre em escala pequena: 23 = 2·11 + 1 é primo seguro, 5 é raiz primitiva e 2 não ($2^{11} \equiv 1 \bmod 23$).

4. Código do algoritmo (DiffieHellman.java)

Implementado em Java 21, na aplicação Spring Boot da disciplina, com `BigInteger` (exponenciação modular com `modPow`) e `SecureRandom` para as chaves privadas. A tela, o controller e os testes estão no repositório: github.com/lucasdu01/UFJF-DCC075-Seguranca (pacote `dh`).

```
package br.ufjf.dcc075.dh;

import org.springframework.stereotype.Service;

import java.math.BigInteger;
import java.security.SecureRandom;
import java.util.List;

import static java.math.BigInteger.ONE;
import static java.math.BigInteger.TWO;

/**
 * Troca de chaves Diffie-Hellman.
 * <p>
 * Públicos: p (primo) e g (raiz primitiva módulo p)
 * Alice: sorteia a (secreto) e envia A = g^a mod p
 * Bob:   sorteia b (secreto) e envia B = g^b mod p
 * Segredo: s = B^a mod p = A^b mod p = g^(ab) mod p
```

```
 * <p>
 * Quem escuta a conversa vê p, g, A e B, mas para calcular s precisaria
 * descobrir a ou b a partir de A ou B: o problema do logaritmo discreto,
 * inviável com p de 2048 bits.
 */
@Service
public class DiffieHellman {

    /**
     * Primo de 2048 bits do RFC 3526 (grupo 14), usado em TLS, SSH e IPsec.
     * É um primo seguro: p = 2q + 1, com q também primo.
     */
    public static final BigInteger P_RFC3526 = new BigInteger(
            "FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD129024E088A67CC74"
            + "020BBEA63B139B22514A08798E3404DDEF9519B3CD3A431B302B0A6DF25F1437"
            + "4FE1356D6D51C245E485B576625E7EC6F44C42E9A637ED6B0BFF5CB6F406B7ED"
            + "EE386BFB5A899FA5AE9F24117C4B1FE649286651ECE45B3DC2007CB8A163BF05"
            + "98DA48361C55D39A69163FA8FD24CF5F83655D23DCA3AD961C62F356208552BB"
            + "9ED529077096966D670C354E4ABC9804F1746C08CA18217C32905E462E36CE3B"
            + "E39E772C180E86039B2783A2EC07A28FB5C55DF06F4C52C9DE2BCBF695581718"
            + "3995497CEA956AE515D2261898FA051015728E5A8AACAA68FFFFFFFFFFFFFFFF", 16);

    /**
     * Menor raiz primitiva módulo P_RFC3526. O gerador g = 2 do RFC NÃO é raiz
     * primitiva: ele gera só o subgrupo de ordem q (2^q ≡ 1 mod p).
     */
    public static final BigInteger G_RAIZ_PRIMITIVA = BigInteger.valueOf(11);

    /** Certeza do teste de Miller-Rabin: chance de erro < 2^-50. */
    private static final int CERTEZA_PRIMO = 50;

    private final SecureRandom random = new SecureRandom();

    /** Uma premissa e se ela foi atendida (exibida na tela com ✓/✗). */
    public record Premissa(String descricao, boolean atendida) {}

    /** Confere as premissas dos parâmetros escolhidos. */
    public List<Premissa> verificar(BigInteger p, BigInteger g, BigInteger a, BigInteger b) {
        boolean primoSeguro = ehPrimoSeguro(p);
        return List.of(
                new Premissa("p é primo seguro (p e q = (p−1)/2 primos)", primoSeguro),
                new Premissa("g é raiz primitiva módulo p", primoSeguro && ehRaizPrimitiva(g, p)),
                new Premissa("1 < a < p−1", noIntervalo(a, p)),
                new Premissa("1 < b < p−1", noIntervalo(b, p))
        );
    }

    /** p = 2q + 1 com p e q primos. Com isso, os únicos fatores primos de p−1 são 2 e q. */
    public boolean ehPrimoSeguro(BigInteger p) {
        BigInteger q = p.subtract(ONE).shiftRight(1);
        return p.isProbablePrime(CERTEZA_PRIMO) && q.isProbablePrime(CERTEZA_PRIMO);
    }

    /**
     * g é raiz primitiva quando suas potências geram todos os valores de 1 a p−1,
     * ou seja, quando a ordem de g é p−1. Isso vale se g^((p−1)/f) ≠ 1 para todo
     * fator primo f de p−1. Para um primo seguro (fatores 2 e q), sobram dois testes:
     * g^2 ≠ 1 e g^q ≠ 1 (mod p).
     * <p>
     * Pré-condição: p é primo seguro. Para um p qualquer de 2048 bits, fatorar p−1
     * seria inviável, e por isso não daria para verificar a raiz primitiva.
     */
    public boolean ehRaizPrimitiva(BigInteger g, BigInteger p) {
        if (g.compareTo(ONE) <= 0 || g.compareTo(p.subtract(ONE)) >= 0) {
            return false;
        }
        BigInteger q = p.subtract(ONE).shiftRight(1);
        return !g.modPow(TWO, p).equals(ONE) && !g.modPow(q, p).equals(ONE);
    }

    /** Sorteia uma chave privada uniforme em [2, p−2] com um gerador criptográfico. */
    public BigInteger gerarChavePrivada(BigInteger p) {
        BigInteger x;
        do {
            x = new BigInteger(p.bitLength(), random); // sorteia e descarta o que cair fora do intervalo
        } while (!noIntervalo(x, p));
        return x;
    }
```

```
    /** Chave pública: g^privada mod p (A para Alice, B para Bob). */
    public BigInteger chavePublica(BigInteger g, BigInteger privada, BigInteger p) {
        return g.modPow(privada, p);
    }

    /**
     * Segredo compartilhado: (chave pública do outro)^privada mod p.
     * <p>
     * A chave pública recebida é validada antes: com A = 1 ou A = p−1, o segredo
     * seria 1 ou ±1 qualquer que fosse a chave privada (ataque de subgrupo pequeno).
     *
     * @throws IllegalArgumentException se a chave pública não estiver em 1 < valor < p−1
     */
    public BigInteger segredo(BigInteger publicaDoOutro, BigInteger privada, BigInteger p) {
        if (!noIntervalo(publicaDoOutro, p)) {
            throw new IllegalArgumentException("Chave pública inválida: deve estar entre 1 e p−1
(exclusive).");
        }
        return publicaDoOutro.modPow(privada, p);
    }

    /** 1 < x < p−1 */
    private boolean noIntervalo(BigInteger x, BigInteger p) {
        return x.compareTo(ONE) > 0 && x.compareTo(p.subtract(ONE)) < 0;
    }
}
```

5. Execução

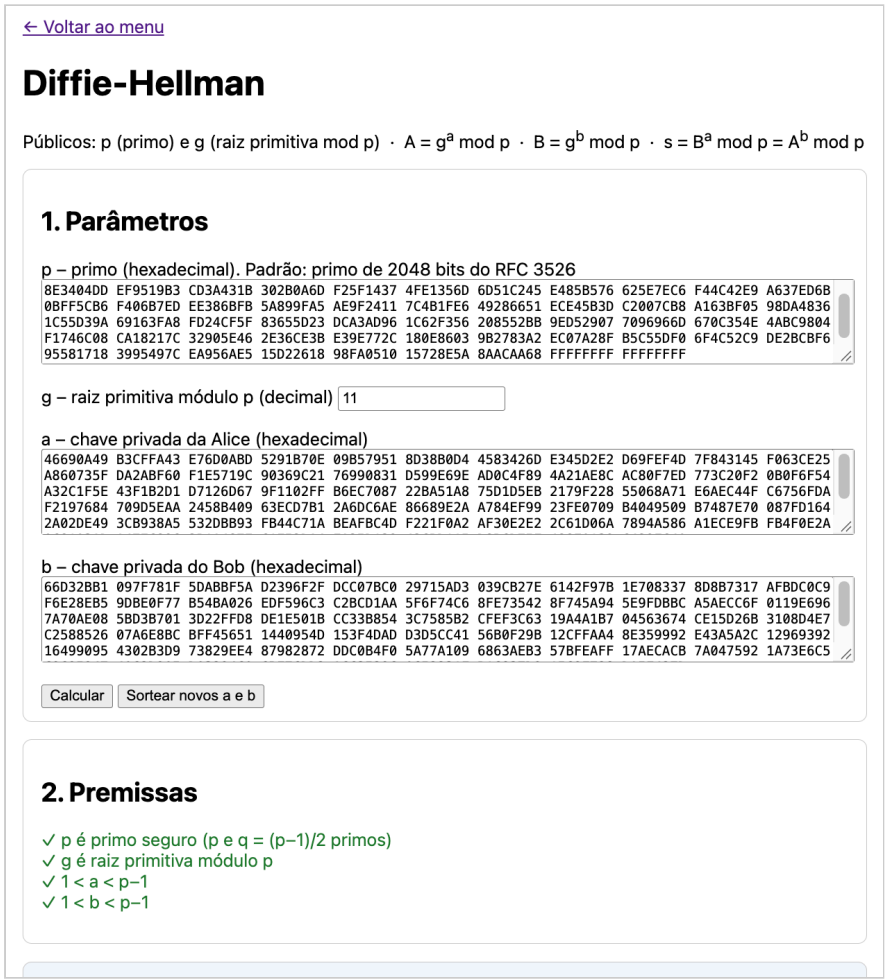


Figura 1 — Tela /dh: parâmetros (p de 2048 bits, g = 11, a e b) e premissas atendidas.



Figura 2 — A e B no canal público; Alice e Bob chegam ao mesmo segredo s.

6. Conclusão e limitações

A implementação cumpre o protocolo: com p primo de 2048 bits e g raiz primitiva, os dois lados obtiveram o mesmo segredo s, que nunca trafegou. O DH puro, porém, tem limites importantes:

- **Man-in-the-middle:** o protocolo não autentica ninguém; um atacante no meio faz uma troca com cada lado e lê tudo. No TLS, os valores do DH são assinados com o certificado do servidor.
- **Tamanho e reuso de p:** o ataque Logjam (2015) quebrou o DH de 512 bits e mostrou que primos de 1024 bits muito compartilhados podem ser pré-computados; hoje, o mínimo é 2048 bits.
- **Raiz primitiva vaza 1 bit:** pelo critério de Euler, qualquer um verifica se A é resíduo quadrático ($A^q \bmod p = 1$) e descobre a paridade de a — e, com as paridades de a e b, se s é resíduo quadrático. Por isso os RFC 3526 e 7919 usam um gerador do subgrupo de ordem q (como g = 2), em que esse bit não vaza.
- **Chave pública maliciosa:** com A = 1 ou A = p−1, o segredo fica previsível; o código valida que $1 < A < p-1$.
- **Uso do segredo:** na prática, s passa por uma função de derivação de chave (como a HKDF) antes de ser usado, por exemplo, como chave do AES.

Referências

[1] W. Diffie, M. Hellman. *New Directions in Cryptography*. IEEE Transactions on Information Theory, 22(6), 1976.

[2] T. Kivinen, M. Kojo. *More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)*. RFC 3526, 2003.

[3] D. Adrian et al. *Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice*. ACM CCS 2015 (ataque Logjam).

[4] NIST. *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*. SP 800-56A Rev. 3, 2018.